

```
+-----+
| TET NETWORK Issue #1 -- Phrack-Style WP v1.1 |
| Fluid P2P Compute-Energy Resource Protocol / AI-Native Sovereign Layer 1 |
+-----+
```

---[TET NETWORK / WHITEPAPER v1.1]-----

Phrack-Style WP -- 2026-05-22 | 2-column monospace | hex chapters

---[TET NETWORK / WHITEPAPER v1.1]-----

+-----+

TET NETWORK

+-----+

```
=====
| Fluid P2P Compute-Energy Resource Protocol |
=====
```

---[TET NETWORK / WHITEPAPER v1.1]-----

Fluid P2P Compute-Energy Resource Protocol

```
=====
| AI-Native Sovereign Layer 1 |
=====
```

---[TET NETWORK / WHITEPAPER v1.1]-----

AI-Native Sovereign Layer 1

Version: Whitepaper v1.1 Draft

Date: 2026-05-21

Author: Steve

Title: Founder-Architect, TET Network Project

Contact: tetsteve@proton.me

Status: Draft for review. Does **not** supersede [WHITEPAPER.md](#) (Genesis v1.0, 2026-04-28) until merged by explicit commit. ^[1]

Implementation

[docs/WHITEPAPER v1.0 GAPS.md](#),

[docs/SOVEREIGN_OS_PHASE0_SPEC.md](#),

[docs/CODEBASE_OVERVIEW.md](#), [docs/STATUS.md](#) ^[2]

Canonical code: tet-core/ (Rust), tet-network/ui/ (Sovereign OS)

references:

```
=====
| Document map |
=====
```

---[TET NETWORK / WHITEPAPER v1.1]-----

Document map

```
+-----+
| Part | Sections | Scope |
+-----+
| **I - Layer 1 Protocol** | $1-$13 | LI mechanics + **Sovereign OS Suite (Phase 0)** |
| **II - Layer 2 Applications** | $14-$16 | Long-horizon primitives (not Phase 0 deliverables) |
| **III - Open Problems & Comparisons** | $17-$19 | Honest gaps, peer comparison, roadmap |
+-----+
```

Phase 0 language: This draft describes a **public testnet / developer preview**, not a production mainnet. Mainnet parameters, REST stability, and

economic schedules remain subject to change until explicitly frozen.

---[TET NETWORK / WHITEPAPER v1.1]-----

+-----+

Part I - Layer 1 Protocol

+-----+

[0x01] 1. Introduction

====[TET NETWORK / WHITEPAPER v1.1]=====

1. Introduction

Cloud AI infrastructure today is not a technical necessity; it is a capital structure. A small set of operators price inference, control availability, and impose correlated failure on every downstream application. Blockchains that treat all nodes as identical hardware competitors reproduce the same concentration under a different scarcity token (ASICs or stake).

TET Network responds at the protocol layer with a single premise: **compute is energy**. Every device that draws electricity can, in principle, contribute verified work or network maintenance. The protocol does not force smartphones to compete under data-center rules. **Context-Aware Adaptive Consensus (CAAC)** assigns roles from hardware reality: high-throughput nodes run **Proof of Compute (PoC)**; constrained edge nodes run **Proof of Relay (PoR)**.

This whitepaper separates what the **testnet ships today** (Part I) from **research-grade application primitives** (Part II) and from **explicit open problems** (Part III). Critique should target the mechanisms in Part I; Part II is directional intent, not a delivery commitment.

0x01.01 1.1 Relation to Genesis v1.0

Genesis Draft v1.0 (2026-04-28) mixed near-term L1 mechanics with long-term vision in a single §12. Version 1.1 **restructures** that material:

- v1.0 §4-§10, §14 → Part I (renumbered)
- v1.0 §12.5-§12.7 → Part II §13-§15
- v1.0 §12.1-§12.4 (RaaS, marketplace, agents, DeFi) → summarized under §3.3 (Phase 0/1 binding surface)
- v1.0 §13 roadmap → §18
- New: §5 implementation phases for the energy peg; §11 four-slot genesis; §13 Sovereign OS; §17 open problems; §18 comparison; §19 roadmap

[0x02] 2. Design Goals

====[TET NETWORK / WHITEPAPER v1.1]=====

2. Design Goals

1. **Post-quantum security from genesis** — ML-DSA (FIPS 204) as the base signature family; hybrid Ed25519 + ML-DSA for wallet transfer auth in Phase 0 (see §7).
2. **Hardware-adaptive consensus** — CAAC routes PoC vs PoR from measured or attested capability, not a single global puzzle.
3. **Thermodynamic economic binding** — Token issuance and inference settlement tied to **verified physical compute energy**, not abstract hash puzzles (§5). This

is the differentiator vs labor markets (Bittensor) and compute-as-a-service brokers (Gensyn).

4. **Optimistic execution with cryptographic dispute resolution** — Sovereign Runtime accepts commitments; fraud challenged in ZK-Court (§8).
5. **Participation without industrial capital** — Edge light clients and PoR roles bound storage and verification cost for consumer hardware (§9).
6. **Honest documentation** — Where code diverges from prose, §17 records the gap; we do not paper over implementation debt.

[0x03] 3. Architecture Overview

====[TET NETWORK / WHITEPAPER v1.1]=====

3. Architecture Overview

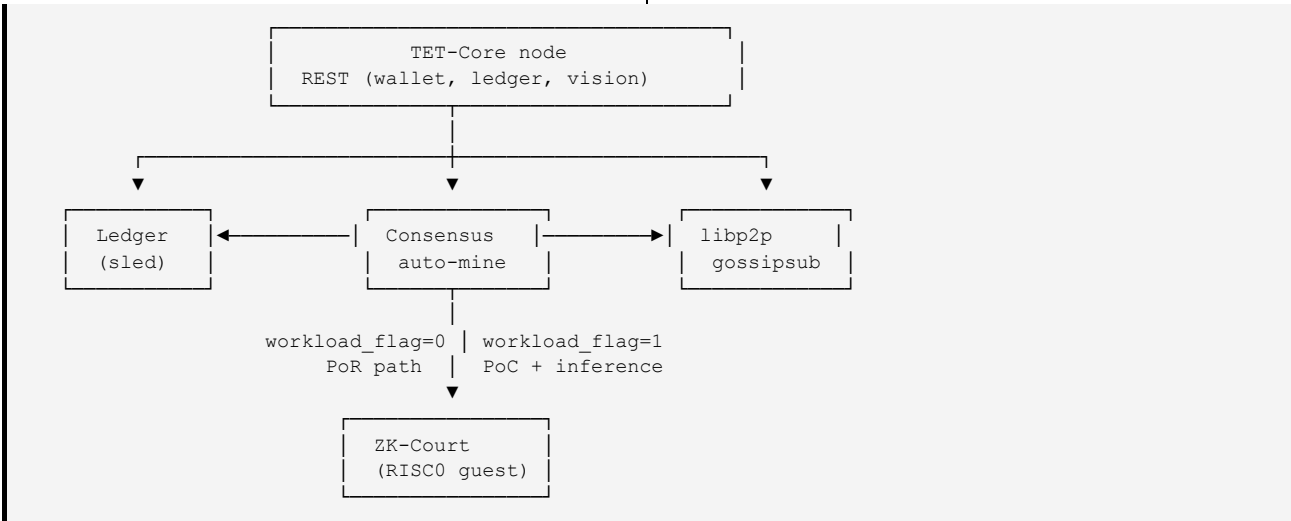
0x03.01 3.1 System components

The canonical node implementation is **tet-core** (TET-Core binary):

Subsystem	Primary modules	Role
Ledger	'ledger.rs', 'genesis.rs'	sled-backed balances, genesis allocation, fees, burns
Consensus / mining	'consensus.rs'	Block production, gossip apply, CAAC leader hints
REST API	'rest/'	Atom HTTP surface for wallet, ledger, vision/ZK-Court
P2P (legacy stack)	'p2p.rs', 'network.rs'	Gossipsub blocks/txs, ledger snapshots
P2P (Phase 1 stack)	'p2p.network.rs', 'p2p.keystore.rs'	libp2p swarm, persistent PeerId
Vision / CAAC	'vision/caac.rs', 'vision/zk_court.rs', 'vision/thermo_genesis.rs'	Roles, disputes, thermodynamic estimates
ZK	'zk_verifier.rs', 'methods/' (RISC0 guest)	Receipt verification, challenge pipeline
Wallet crypto	'wallet.rs', 'quantum_shield.rs'	BIP39 → Ed25519 + ML-DSA hybrid messages

Chain binding: Hybrid signed messages embed `chain_id` and `genesis_hash`. The canonical genesis hash is computed in `tet-core/src/genesis.rs` (single source of truth); `ledger.rs` and `wallet.rs` delegate to it.

0x03.02 3.2 CAAC + PoC + PoR + ZK-Court (control plane)



0x03.03 3.3 Phase 0 / Phase 1 application surface (binding scope)

The following v1.0 §12.1–§12.4 capabilities remain in scope for testnet and Phase 1, but are not renumbered as separate whitepaper chapters in v1.1:

Application	Phase 0 status	Notes
Decentralized inference marketplace	**Partial**	REST + Sovereign OS UI; thermodynamic pricing via §5 Phase 0 approximation
Enterprise / AI workload transactions	**Partial**	'workload_flag=1' mempool path
Agent / M2M clients	**Partial**	'tet-agent-sdk/'; Agent-Gate is Part II
Native L2 RaaS	**Research**	Not operational on testnet

Part II §13–§15 (World Brain, Sentient Assets, Agent-Gate) remain **explicitly out of Phase 0**.

[0x04] 4. Proof of Compute (PoC) and Proof of Relay (PoR)

---[TET NETWORK / WHITEPAPER v1.1]---

4. Proof of Compute (PoC) and Proof of Relay (PoR)

0x04.01 4.1 Proof of Compute (PoC)

Nodes with sufficient GPU/CPU throughput are classified as **PoC producers**. They:

- Execute AI inference or heavy compute tasks off the hot ledger path.
- Submit cryptographic commitments to results (hashes, journals).

- Accept optimistic settlement during a **challenge window**; disputes escalate to ZK-Court (§8).

Rewards scale with **verified thermodynamic work** (§5), not with hash rate on a meaningless puzzle.

Implementation: `worker_daemon.rs`, `vision/caac.rs`, `consensus.rs` (AI workload blocks). PoC eligibility uses CAAC worker records and role flags.

0x04.02 4.2 Proof of Relay (PoR)

Constrained devices (mobile, IoT) participate as **PoR** nodes:

- Propagate blocks, transactions, and signed ledger snapshots.
- Perform lightweight ML-DSA / hybrid signature verification.
- Do **not** execute full inference workloads (`workload_flag=1` rejected on PoR routing).

Economic rewards are smaller than PoC but non-zero, preserving the “universal miner” design goal.

Implementation: `p2p.rs` gossip paths, `network.rs` ledger topic, edge verification in wallet/ledger read paths.

0x04.03 4.3 Workload flag (fluid transaction routing)

Transactions carry a binary `workload_flag`:

Flag	Semantics	Routed to
'0'	Value transfer / maintenance	PoR-eligible validation
'1'	AI inference / compute request	PoC producers only

Misrouting flag-1 work to edge nodes is rejected at the protocol layer (v1.0 §6, preserved in implementation via consensus and mempool filters).

[0x05] 5. Energy Peg – Thermodynamic Reward Model

---[TET NETWORK / WHITEPAPER v1.1]---

5. Energy Peg – Thermodynamic Reward Model

0x05.01 5.1 Vision: the Sovereign Peg

Claim (strategic): TET pegs each unit of circulating supply to **verified physical compute energy** through a worker-specific thermodynamic efficiency term

$\eta(W_i)$. Unlike labor-token markets that reward stake-weighted opinions, and unlike pure compute rental APIs that price hours without a chain-level energy conservation law, TET binds issuance to $\eta(W_i) \cdot C(t_i)$ aggregated over verified tasks, normalized by network difficulty $D(t)$.

The v1.0 continuous-time statement:

$$R(T) = \sum_{i \in \text{verified_tasks}(T)} [\eta(W_i) \cdot C(t_i)] / D(t)$$

Where:

- $\eta(W_i)$ – thermodynamic efficiency of worker i * (energy out per unit useful compute, or equivalent joules attributed to verified work).
- $C(t_i)$ – network compute price at task time t_i .*
- $D(t)$ – dynamic difficulty / scarcity regulator (analogous in role to mining difficulty, but tied to compute-energy targets).

We do not weaken this claim in v1.1. We document **how each phase approximates η** and where formal η remains open (§17.1).

0x05.02 5.2 Formal $\eta(W_i)$ – deferred

A complete definition of $\eta(W_i)$ under adversarial hardware impersonation, cross-vendor GPU counters, and mobile enclave attestations is **not closed in this draft**. §17.1 lists required assumptions. Until then, phases below use **auditable proxies**.

0x05.03 5.3 Phase 0 implementation (current testnet)

Code path: `tet-core/src/vision/thermo_genesis.rs`

The ledger settlement uses a **discrete approximation** aligned with whitepaper §4.2 engineering notes:

$$R_{\text{micro}} = (C_{\text{flops}} / E_{\text{joules_per_flop}}) \times T \times \text{scale}$$

Symbol	Meaning	Source
'C_flops'	Declared inference FLOPs	Task / receipt metadata
'E_joules_per_flop'	Energy proxy **E** (J/FLOP), env 'TET_JOULES_PER_FLOP'	Operator-tunable default '1e-12'
'T'	Network difficulty	'NetworkDifficulty', env 'TET_NETWORK_DIFFICULTY_GAMMA'
'scale'	Maps dimensionless ratio → Stevemom micro	'TET_THERMO_STEVEMON_MICRO_SCALE'

η in Phase 0: Approximated indirectly via **hardware fingerprint class** (CPU vs GPU vs specialized) in CAAC weighting (vision/caac.rs) and static env efficiency – **not** per-device power telemetry.

Gap: This is $(C/E) \times \Gamma$, not the full $\Sigma[\eta \cdot C]/D$ integral. The gap is explicit in \$17.1 and \$17.7 (notation reconciliation).

0x05.04 5.4 Phase 1 target

– η computed from **CAAC fingerprint + measured compute** (timing micro-tasks, FLOPs/second bands, attested GPU class).

– Bind optimistic inference receipts to thermodynamic **R_expected** used in slash economics (\$12).

– Unify discrete thermo_genesis outputs with wallet-visible fee displays (\$11).

0x05.05 5.5 Phase 2 target

– η pegged to **hardware-attested power telemetry** (TPM / secure enclave / datacenter PDU APIs where available).

– Cross-check on-chain R against physical energy invoices for auditability.

0x05.06 5.6 AI inference settlement split (related economics)

Separate from transfer fees (\$11), AI inference settlement splits thermodynamic **R_micro 50/50** between the worker pool and protocol burn, at every height (fees::FeeKind::AiInference).

Corrected 2026-09-17. Earlier drafts stated this 50/50 but the code did not implement it. A Genesis Epoch multiplier computed $\text{pool} = (R/2 \times 5).min(R)$, which saturates – $2.5 \times R$ clamped to R – so for all 1,300,000 Genesis Epoch blocks the real split was **100% pool / 0% burn** and the burn never occurred. The multiplier is removed and the split is now a true 50/50 (docs/FEE_SPEC.md \$2.3).

Enterprise AI **utility** settlement is a different schedule: 20% network fee on gross → **80% worker / 15% treasury / 5% burn** (\$2.2 of the fee spec).

[0x06] 6. Context-Aware Adaptive Consensus (CAAC)

=====[TET NETWORK / WHITEPAPER v1.1]=====

6. Context-Aware Adaptive Consensus (CAAC)

CAAC is the routing and weighting layer that assigns each node an operational role from hardware and network context. v1.0 \$4 content is preserved here as the **core working claim** of the project.

0x06.01 6.1 Role assignment

1. **Probe** – Static and micro-benchmark signals (GPU name, memory, optional timing tasks).

2. **Classify** – PoC vs PoR vs fallback weights.

3. **Leader election** – Hash- or weight-based producer selection for block intervals (consensus.rs, CAAC records in ledger).

0x06.02 6.2 PoC weight factors

High-performance nodes earn higher **CAAC weight** from:

- Proof-of-capacity signals (GPU tier, memory),
- Historical latency and availability,
- Verified inference deliveries (feeds ZK-Court and thermodynamic history).

0x06.03 6.3 PoR weight factors

Edge nodes earn weight from:

- Successful gossip propagation,
- Signature verification throughput,
- Uptime on relay paths.

0x06.04 6.4 Implementation status

Feature	Status
Worker records in ledger	**Implemented**
Static hardware probes	**Partial**
Probabilistic timing fingerprinting (\$10)	**Partial**
Full autonomous reclassification every epoch	**Open** (\$17.5)

[0x07] 7. Post-Quantum Cryptography

---[TET NETWORK / WHITEPAPER v1.1]---

7. Post-Quantum Cryptography

0x07.01 7.1 ML-DSA from genesis

TET adopts **ML-DSA** (FIPS 204, module-lattice signatures) as the protocol-family PQC scheme. Migration-from-ECDSA is intentionally avoided.

Surface	Level	Why
Browser wallet (all user signatures)	**ML-DSA-44**	'tet-pqc-wasm' builds level 44 only
Node default for mnemonic-derived keys	**ML-DSA-44**	wallet::active_mldsa_mode() was 65 until 2026-09-17, which silently disagreed with every wallet on the network
Node auxiliary keystore	ML-DSA-65	pqc_keystore.rs - server-side operations, not a user identity
'tet-signer' (macOS Keychain / Secure Enclave)	ML-DSA-65	independent device-bound key, randomly seeded, **not** mnemonic-derived

Operators may select **65** or **87** node-side via `TET_MLDSA_SECURITY_LEVEL`.

Level inference on verification.
`wallet::verify_mldsa_b64` does not take a level argument. It

infers the parameter set from the **public-key length** (1312 / 1952 / 2592 bytes → 44 / 65 / 87)

and accepts all three. This is why the level split above went unnoticed: mixed-level signatures

verify fine. It also means the effective security level of any given signature is whatever the

signer chose, not a protocol-enforced floor. Pinning a minimum level is a consensus change and is

```
tet xfer hybrid vl|chain_id=...|genesis_hash=...|to=...|amount_micro=...|nonce=...|mldsa_pubkey_b64=...
```

`genesis_hash` must match
`genesis::expected_genesis_hash_from_env()` on the node.

0x07.03 7.3 Node ML-DSA keystore

`pqc_keystore::ensure_node_mldsa_keystore` provisions node-level **ML-DSA-65** keys under the DB directory for server-side operations. These are distinct from the user wallet identity (\$7.1) and are never used to authorise a transfer.

Implementation: `quantum_shield.rs`, `tet-pqc-wasm/`, the `dilithium-rs` crate in `wallet.rs`.

Parameter set actually shipped in Phase 0

Phase 0 ships ML-DSA-44 (NIST security level 2) for all user wallets. This is not a simplification of a higher level — it is what the code does, and this section previously implied otherwise.

deferred to the Phase 1 genesis.

0x07.02 7.2 Phase 0 hybrid wallet auth

Wallet transfers require **both**:

- **Ed25519** over a deterministic UTF-8 message (BIP39 seed bytes [0..32] → signing key), and
- **ML-DSA** over the same message bytes.

This hybrid path is what Sovereign OS uses for POST `/wallet/transfer`. See

`wallet.rs::transfer_hybrid_auth_message_bytes` and UI `ed25519_tet.ts`.

Chain binding fields in the message:

0x07.04 7.4 Quantum threat model

ECDSA-secured chains face retroactive migration risk under Shor-capable adversaries. TET assumes ML-DSA parameter sets aligned with NIST guidance; parameter agility for future standards remains an operational concern, not a Phase 0 blocker.

[0x08] 8. ZK-Court (Lazy Evaluation)

====[TET NETWORK / WHITEPAPER v1.1]=====

8. ZK-Court (Lazy Evaluation)

0x08.01 8.1 Threat addressed

A malicious PoC node may submit **fabricated inference** without executing the model. ZK-Court provides **lazy evaluation**: optimistic acceptance during a challenge window, then cryptographic replay.

0x08.02 8.2 Mechanism

Backend	Status
RISC Zero ('methods/', 'worker_daemon.rs')	**Implemented** (requires built guest ELF)
SP1	**Not integrated** (\$17.2)

0x08.04 8.4 Mainnet safety

TET_MAINNET=1 forbids mock ZK paths (zk_verifier.rs, main.rs panic on TET_ALLOW MOCK_ZK=1). Dev-only MOCKJ1: / MOCKZC1: prefixes are rejected on mainnet.

[0x09] 9. Network Layer (libp2p)

====[TET NETWORK / WHITEPAPER v1.1]=====

9. Network Layer (libp2p)

0x09.01 9.1 Transport and identity

- **Transport**: TCP + Noise XX + Yamux (Phase 1 swarm in p2p_network.rs).

Topic constant	Purpose
'BLOCKS_TOPIC'	Block propagation ('p2p.rs')
'TXS_TOPIC'	Transaction gossip
'AI_WORKLOAD_TOPIC'	AI workload announcements
'TET_LEDGER_TOPIC'	Signed ledger snapshot replication ('network.rs')
'nexus-inference-v1'	Inference gossip ('p2p_network.rs')

0x09.03 9.3 Sync and testnet operations

Sprint 1 added **chain catch-up** (sync.rs) and sync-gated auto-mine. Multi-node docker compose (tet-core/docker-compose.yml, scripts/start-network.sh) is the recommended Phase 0 operator path. See docs/RUNNING_A_NODE.md.

1. PoC records delivery + commitment (vision/zk_court.rs).

2. Challenge window opens (TET_ZK_COURT_CHALLENGE_MS, default 24h).

3. Challenger posts bond; pipeline runs **RISC Zero** guest prove when NEXUS_GUEST_ELF is non-empty.

4. **Guilty** if verified receipt shows journal ≠ commitment.

5. **Slash** on guilty verdict (\$12).

0x08.03 8.3 Prover backends

0x08.05 8.5 REST surface (operator)

- POST /v1/vision/zk-court/challenge – full pipeline

- Params JSON exposes **whitepaper_alignment** block for operators (WHITEPAPER_v1.0_GAPS.md §6)

- **Identity**: Persistent Ed25519 libp2p keypair in libp2p_keypair.bin under DB dir (p2p_keystore.rs). Boot banner logs PeerId for bootnode wiring.

- **Discovery**: mDNS, Kademlia, identify, autonat, relay (compose-dependent).

0x09.02 9.2 Gossip topics (representative)

0x09.04 9.4 Cockroach Doctrine (resilience)

If data-center PoC clusters fail, PoR mesh continues header propagation and ledger continuity; inference throughput degrades, chain liveness does not halt. This remains a **design property**; planetary-scale PoR counts are not yet demonstrated publicly.

[0x0A] 10. Hardware Fingerprinting (Sybil Resistance)

---[TET NETWORK / WHITEPAPER v1.1]---

10. Hardware Fingerprinting (Sybil Resistance)

0x0A.01 10.1 Attack

Adversary claims PoC-class rewards while running low-tier hardware.

0x0A.02 10.2 Defense (v1.0 §14.2, preserved)

Probabilistic hardware fingerprinting: non-deterministic micro-tasks whose timing distributions are silicon-specific. Emulation must reproduce

physical timing at scale.

0x0A.03 10.3 Implementation

vision/caac.rs — static probes (GPU name, memory heuristics) and limited micro-benchmark hooks. Full probabilistic schedule is **partial** (§17.5).

0x0A.04 10.4 Phase 2

Integration with secure enclaves (see Part II §15 Agent-Gate pillar 3) for presence proofs without per-request ZK.

[0x0B] 11. Tokenomics

---[TET NETWORK / WHITEPAPER v1.1]---

11. Tokenomics

0x0B.01 11.1 Supply cap

Unit	Definition	On-chain representation
TET	Human-facing token	—
Stevemon	Sub-unit name in code/comments	1 TET = **10** Stevemon
micro-TET / *_micro* fields	Integer ledger amounts	'u64' micro-units (= Stevemon atoms)

REST stability (Phase 0): Field names may change pre-mainnet (amount_tet as f64 vs amount_micro as u64 in signing). §17 documents API gaps.

Maximum supply: 10,000,000,000 TET (MAX_SUPPLY_MICRO in ledger.rs / genesis.rs).

0x0B.02 11.2 Denomination table

0x0B.03 11.3 Genesis four-slot allocation

Genesis mint is binding via tet-genesis-v1 hash (genesis.rs). Four logical slots:

Slot	Wallet id	Share	Phase 0 mint
Worker pool	'000_0001' ('WALLET_WORKER_POOL')	**50%** (5B TET)	Full tranche to locked system pool
Founder	'TET_GENESIS_FOUNDER_WALLET_ID' / 'TET_FOUNDER_WALLET' (64-hex Ed25519 id)	**25%** (2.5B TET)	Credited at genesis; **vesting / unlock schedule open** (§17.3)
Treasury	'TET_TREASURY_ADDRESS' (required 64-hex)	**25%** (2.5B TET)	Credited at genesis
Protocol reserve	'000_0003' ('WALLET_PROTOCOL_RESERVE')	**0%** (placeholder)	**Zero by design** in Phase 0

Legacy sentinel **000..0002** (WALLET_ECOSYSTEM) receives **no mint** post-Phase 2B; treasury tranche moved to env-configured address (WHITEPAPER_v1.0_GAPS.md §9).

Repurposing reserve (non-zero mint or address change) alters genesis_hash → **chain-incompatible** without new genesis ceremony.

— Emergency operational grants

— Ecosystem grants approved by on-chain proposal

— Potential buyback / burn policy (subject to governance)

Phase 0: allocation remains **zero** intentionally; nodes still commit reserve_micro=0 in genesis hash.

0x0B.04 11.4 Protocol Reserve purpose (Phase 1+)

The reserve slot exists in the hash payload for forward compatibility. Intended uses (governance-defined, not yet allocated):

— Security bug bounties

0x0B.05 11.5 Sovereign OS micropayments (Phase 0 — Steve decisions locked)

Protocol fees for **Tmail** and related Sovereign OS actions (spec: [SOVEREIGN_OS_PHASE0_SPEC.md](#) Appendix C). Amounts are **testnet defaults** until mainnet freeze. ^[3]

Action	Charge	Ledger field	Notes
Tmail send	**1 Stevemom** (= 1 µTET)	'fee_paid_micro = 1'	~free UX; spam gate; Phase 0.1 may raise
Tmail Pin	**1000 Stevemom**	'pin_stake_micro = 1_000'	Persistent thread beyond 5-msg UI cap
Anonymous escrow	**1 TET**	'1_000_000' µTET	Minimum stake; 24h auto-settle
File share / pin	TBD	-	Size-dependent Fee **Phase 0.1**

Fee disposition (Tmail/Pin/Anonymous protocol fees): Faucet (testnet): 100 TET per day per IP on public seed – not a protocol fee; operator policy.

50% treasury / 50% burn – same deflationary story as \$11.6, routed to treasury address and burn sink at settlement.

0x0B.06 11.6 Transfer fee schedule (implemented)

Parameter	Value	Code
Maintenance fee	**1%** of gross transfer	'PROTOCOL_MAINTENANCE_FEE_BPS = 100'
Fee split	**50%** to worker pool, **50%** burned	'ledger.rs' transfer settlement

Example (Phase 0 E2E): send 1,000,000 micro (1 TET) → fee_micro = 10,000, net_micro = 990,000.

Note: This is the wallet transfer path (POST /wallet/transfer). Mempool POST /ledger/transfer uses a different envelope (SignedTxEnvelopeV1).

thermodynamic 50/50 worker/burn split (\$5.6). Unified narrative for v1.1 mainnet docs remains open (\$17.7).

0x0B.08 11.8 Worker pool emission

0x0B.07 11.7 Deflationary burn (vision vs transfer path)

v1.0 \$11.2 states 50% of fees burn under sustained use. Implementation matches this for standard wallet transfers. AI inference economics additionally use

The 50% worker pool tranche is minted at genesis to a locked address without a private key. Ongoing PoC/PoR emissions from pool to producers follow coinbase and settlement rules in ledger.rs. Emission curve shape (constant vs decreasing) is open (\$17.4).

[0x0C] 12. Economic Security (Slash Model)

====[TET NETWORK / WHITEPAPER v1.1

Code: vision/zk_court.rs, ledger.rs

12. Economic Security (Slash Model)

0x0C.01 12.1 Lazy-evaluation fraud (ZK-Court)

For proven inference fraud during the challenge window, the implementation executes:

slash_worker_bond_to_ecosystem_all(worker)

i.e. 100% of liquid worker bond is burned to the ecosystem sink. This matches v1.0 \$14.1 and \$5.1 “confiscation, not a fine.”

Offense class	Phase 0 rule	Future
ZK-Court inference fraud	**100% bond slash**	Keep
Other Byzantine behaviors (double-sign, invalid blocks, etc.)	Ad hoc / partial	**Cap at min(bond, λ·R_expected)** Phase 1

Today λ and R_expected are telemetry for disputes; they do not cap ZK-Court slashes. See WHITEPAPER_v1.0_GAPS.md §3.

at mainnet freeze.

0x0C.03 12.3 Rationality argument

For fraud with 100% slash, expected gain must be < bond. For future capped offenses, the inequality S > R_expected should be enforced by parameter choice

0x0C.02 12.2 Parametric model (other offense classes – Phase 1+)

v1.0 \$14.3 defines slashable collateral:

$$S = \lambda \cdot R_{\text{expected}}$$

with λ default 100 (TET_SLASH_LAMBDA_MULTIPLIER), and R_expected stored on artifacts from settlement.

v1.1 choice (Option A – honest to code):

0x0C.04 12.4 Founder and treasury operational security

Founder premine is subject to unlock policy (\$17.3). Treasury address is env-driven for testnet; production treasury migration requires coordinated genesis or transfer policy.

[0x0D] 13. Sovereign OS Suite (Phase 0)

---[TET NETWORK / WHITEPAPER v1.1]---

13. Sovereign OS Suite (Phase 0)

Phase 0 deliverable. Sovereign OS is the user-facing surface of TET Network: not a wallet tab, but a daily-use **desktop environment** backed by **tet-core** and **libp2p**. Full technical spec: [SOVEREIGN OS PHASE0 SPEC.md](#).

0x0D.01 13.1 Philosophy

- **TET UI ≠ wallet** - Users live in an OS metaphor (apps, windows, taskbar), not a single "Send Coins" page.

- **Local-first, node-backed** - Browser talks to the user's **tet-core** REST API; **libp2p** runs in the node, not in the browser.

App	Role
Wallet	Hybrid Ed25519 + ML-DSA transfers; genesis-bound auth
Tmail	Encrypted messaging (\$13.4)
Files	Local mailbox, libp2p P2P transfer, optional paid replication
Explorer	Block / transaction browse
Mini-apps	Calculator (TET/USD/JPY), Clock (block height), Notes (encrypted local)

Worker inference UI is **hidden** in Phase 0 (SHOW_WORKER_TAB=true for developers). Productized earn path → Phase 0.5.

#	Feature	Phase 0 mechanism
1	**Basic E2EE**	X25519 + CRYSTALS-Kyber-768 (Round-3) + ChaCha20-Poly1305 ('tet-core/src/e2ee.rs')
2	**Time-lock**	**Stake-scheduled** 'release_at_ms' (nodes enforce decrypt policy); VDF upgrade \$17.8
3	**Burn-after-read**	Read receipt → gossip revoke; **best-effort** (\$17.9)
4	**Anonymous sender**	Anchor + ephemeral + RISC0 ownership proof; **1 TET** escrow
5	**5-msg window + Pin**	UI cap; **1000 Stevemom** stake for persistence

Gossip topic: /tet/v1/tmail on the block-plane libp2p swarm. Ciphertext is **not** stored on-ledger; fees and audit metadata are.

```

Anchor wallet (persistent, BIP39)
  |
  +-- fund + escrow (1 TET minimum)
  |
  v
Ephemeral wallet (per-send/session) --ZK proof--> "anchor owns ephemeral" (RISC0 guest)
  |
  +-- send Tmail (gossip shows ephemeral only)
  |
  v
Receiver sees anonymous sender; third parties cannot link to anchor
  |
  +-- Anchor-only audit trail (REST, hybrid-signed) - voluntary disclosure path

```

- **24h auto-settle** returns unused escrow to anchor unless dispute.

- **L1 before apps** - Public testnet (seed, faucet, Docker) ships in **Sprint 4** before Tmail (see \$19.1).

0x0D.02 13.2 Desktop shell - "Inspired by 1990s desktop OS"

Visual design uses open **98.css** styling: gray bevel chrome, draggable windows, taskbar, Start menu, boot animation. Marketing and About copy:

"Inspired by 1990s desktop OS" - no Microsoft® trademarks; not affiliated with or endorsed by Microsoft.

0x0D.03 13.3 Core applications (Phase 0)

Onboarding: docker compose up launches **node** + **UI** (mandatory path for general users).

0x0D.04 13.4 Tmail - five capabilities

Marketing discipline: Public "world-first" claims for time-lock, burn, and anonymous modes require **acceptance tests AT-3, AT-4, AT-5** on the public testnet (Steve decision).

0x0D.05 13.5 Anonymous Mode architecture

- Misuse deterrence: high escrow + ZK-Court slash linkage (\$12).

0x0D.06 13.6 Phase 0 operations and ship target

Item	Policy
Ship target	**2026-09-15** (feature freeze **2026-08-31**, 2-week polish)
Public seed	**1x** Hetzner EU (~\$5/mo) pre-ship; 2nd node when traffic warrants (SPOF accepted)
Faucet	**100 TET / day / IP**
Sprint plan	S4 Foundation → S5 Tmail protocol → S6 E2EE+shell → S7 time-lock/burn/pin → S8 Anonymous+ZK → S9 Files → S10 mini-apps → S11 QA

=====
 ---[TET NETWORK / WHITEPAPER v1.1]-----

Part II – Layer 2 Applications (Future Vision)

Not Phase 0 deliverables. The following sections describe architectural intent. Implementation depends on resolving open problems in federated

learning, on-chain inference economics, and machine-to-machine settlement.

[0x0E] 14. World Brain (Neural State Transitions)

=====
 ---[TET NETWORK / WHITEPAPER v1.1]-----

14. World Brain (Neural State Transitions)

During idle periods, PoC nodes contribute spare compute to **federated fine-tuning** of a shared base model. The chain state becomes a living, tamper-

resistant model artifact refined by privacy-preserving edge telemetry.

Dependencies: Part I CAAC + ZK-Court operational at scale; privacy and model-governance open problems.

Status: research / not implemented.

[0x0F] 15. Sentient Assets (Smart Contracts 2.0)

=====
 ---[TET NETWORK / WHITEPAPER v1.1]-----

15. Sentient Assets (Smart Contracts 2.0)

Contracts embed inference over context – wallets that reason about fraud patterns, assets that negotiate price from live ecosystem data – built

atop World Brain state transitions.

Status: research / not implemented.

[0x10] 16. Agent-Gate (Machine-to-Machine Economy)

=====
 ---[TET NETWORK / WHITEPAPER v1.1]-----

16. Agent-Gate (Machine-to-Machine Economy)

An M2M API gateway imposing **economic friction** on autonomous agent swarms without degrading human UX:

1. **Invisible UX** – Sovereign OS local agent spends micro-TET in background.

2. **State channels** – high-frequency agent payments settle net on L1 daily.

3. **Hardware-enclave PoR** – presence proofs via Secure Enclave / TrustZone without per-request ZK.

Status: research / not implemented; overlaps \$10 enclave roadmap.

=====
 ---[TET NETWORK / WHITEPAPER v1.1]-----

Part III – Open Problems and Comparisons

[0x11] 17. Open Problems

---[TET NETWORK / WHITEPAPER v1.1]

17. Open Problems

Each item is stated bluntly; solutions are not claimed unless noted.

0x11.01 17.1 Formal definition of $\eta(W_i)$ under adversarial hardware impersonation

We lack a closed-form $\eta(W_i)$ that is simultaneously measurable on commodity hardware, resistant to emulator timing attacks, and composable into $R(T)$. Phase 0 uses $(C/E) \times T$ proxies. **Status: open.**

0x11.02 17.2 SP1 prover integration

ZK-Court pipeline is RISC Zero-only when ELF is present. SP1 is cited in v1.0 but not wired in zk_verifier.rs. **Status: open** (RISC0 partial).

0x11.03 17.3 Founder unlock schedule (cliff vs linear vest)

Founder tranche mints at genesis; much of founder balance may be **locked** in ledger policy. Cliff vs linear vest for mainnet is undecided. **Status: open.**

0x11.04 17.4 Worker pool emission curve

Genesis mints 5B TET to locked pool; ongoing emission rate shape (constant vs decay) not finalized. **Status: open.**

0x11.05 17.5 CAAC hardware fingerprinting attack model

Timing micro-tasks are described; formal security game (emulator, FPGA, cloud GPU posing as mobile) is not written. **Status: open** (implementation partial).

Concrete instance of this gap (2026-09-20). POST /v1/vision/caac/complete accepts

client_latency_ms from the caller and derives the PoC/PoR role from it

(vision::caac::role_from_latency_ms). The resulting record feeds

LedgerCaacWeightProvider::consensus_weight (consensus.rs:200), which sets leader-election weight when TET_CONSENSUS_LEADER_MODE=caac — the value .env.mainnet.example ships.

The endpoint now requires a hybrid Ed25519 + ML-DSA signature, which closes **impersonation**: you

can no longer write a role record for a wallet you do not control. It does **not** close

self-elevation. A node can honestly sign its own client_latency_ms: 0, be classified PoC, and

take weight 100 plus a latency bonus of up to 1000 — while a truthful slow node takes 25.

The signature proves who is speaking, not that the number is true. Closing this needs

server-measured or hardware-attested latency, which is the formal model this section is about.

Tracked for the ceremony in PHASE 1 GENESIS SPEC.md §2.3. ^[4]

0x11.06 17.6 Cross-chain bridge design (Phase 1 interoperability)

No canonical bridge spec for ETH/SOL/BTC custody. **Status: open.**

0x11.07 17.7 Slash magnitude and economics notation reconciliation

\$12 documents 100% slash for ZK fraud vs \$14.3 $\lambda \cdot R_{\text{expected}}$ parametric model. AI 50/50 thermodynamic split vs global fee-burn wording differs. **Status: partial** (documented in GAPS doc; code uses Option A).

0x11.08 17.8 Time-lock cryptography upgrade

Phase 0: Time-lock delivery uses **stake-scheduled release** (release_at_ms in signed envelope; nodes refuse early decrypt). Trust is **social/protocol-enforced**, not wall-clock cryptographic.

Phase 0.1+: Verifiable Delay Function (VDF) or threshold decryption for **trustless** time-lock. Class-group VDF evaluation is research-track (see SOVEREIGN OS PHASE0 SPEC.md §A.2). ^[5]

Status: open (VDF); Phase 0 path **implemented by policy.**

0x11.09 17.9 Burn-after-reading cryptographic guarantee

Phase 0: Burn is **best-effort**: cooperating peers delete ciphertext after read-receipt gossip; malicious archivers may retain encrypted blobs. UI copy states this explicitly.

Phase 1+: Forward secrecy sessions + cryptographic key destruction proofs; possible CRDT-wide revocation research.

Status: **open** (strong guarantee); Phase 0 path **best-effort**.

0x11.0A 17.10 REST API completeness

POST /wallet/transfer lacks tx_hash / block confirmation fields; amount_tet f64 vs signed amount_micro u64. **Status:** **open** (Phase 1).

0x11.0B 17.11 Multi-node sync at public testnet scale

Catch-up driver exists; 72h public testnet soak not completed. **Status:** **partial** (docs/SYNC_ISSUE.md).

Class	Examples	TET reuse
GPU PoW miners	GRIN, Ergo, Ethereum Classic, other general-compute-friendly PoW	**Candidates** for idle-time switching
SHA-256 ASIC miners	Bitcoin core hashrate	**Excluded** - hardware incompatible with general AI inference

Mechanism (speculative): idle-time auto-switching between PoW mining and TET worker mode, gated by **real-time profitability oracles** (hashprice vs thermodynamic inference reward R_micro, \$5.3) and CAAC role classification (\$6).

Open questions:

- **Incentive compatibility** - Does switching dominate single-protocol mining in expectation, or does pool lock-in / variance dominate?
- **Switching latency and cooling** - Thermal cycles and driver reload cost when bouncing between PoW kernels and inference runtimes.
- **Anti-cheating / fingerprinting** - CAAC hardware attestation (\$10) must remain stable across protocol switches; emulator timing games may worsen.
- **Mining pool economics** - Integration with PPS, FPPS, and stratum-style contracts without violating pool ToS or double-spend of hashrate commitments.

Status: research / future direction. **Phase 0 does not ship this.** Phase 1+ exploration only; no commitment to dual-mining productization.

0x11.0E 17.14 Browser-embedded node architecture

Most IIs (Bitcoin, Ethereum) gate participation on running external node software. Because the Sovereign OS shell (\$13) already runs entirely in the browser, an open question is whether a **light TET node** can run **in-tab** - letting end-user devices join the network natively, without operator-run servers.

0x11.0C 17.12 Dual genesis_hash class of bugs (resolved 2026-05-20)

Wallet vs ledger genesis hash divergence broke all hybrid transfers. Fixed via genesis.rs. **Status:** **closed** in testnet; recorded as process lesson.

0x11.0D 17.13 Mining hardware reuse for AI inference

Bitcoin and PoW altcoin networks consume on the order of **~150 TWh annually** (order-of-magnitude industry estimate; exact figure varies by source and year) with **no useful computational output** beyond hash puzzle completion. TET's CAAC worker model could, in principle, **dual-purpose GPU miners** during low-profit windows to earn **AI inference rewards** on the same hardware class already deployed for PoW.

Scope (candidates vs excluded):

A libp2p **WebRTC transport** plus a light-client verification path could embed peer discovery, header sync, and transaction submission directly in the page. The directional goal: as user devices proliferate, Foundation-operated infrastructure trends toward zero.

Open questions:

- **Transport maturity** - Production-readiness of libp2p WebRTC (NAT traversal, signaling, connection churn) inside browsers.
- **State budget** - Holding ledger headers/state within IndexedDB quota limits.
- **Mobile cost** - Battery and data overhead of sustained P2P on phones.
- **Trust model** - Light-client assumptions vs full-node verification; fraud-proof or sampling strategy.
- **Bootstrap** - Peer discovery from a cold browser tab with no inbound connectivity.

Status: research / future direction. **Phase 0 does not ship this.** Phase 2+ exploration, contingent on libp2p WebRTC maturation; multi-year horizon, no commitment.

0x11.0F 17.15 AI-native smart contracts

Most contract platforms treat AI as an **external oracle**: inference runs off-chain and a result is posted back. Because TET's CAAC consensus (\$6) already settles AI inference as a first-class network operation, an open question is whether

programmable contracts (cf. Sentient Assets, \$15) could **invoke inference natively** — making an `infer(...)` call a primitive alongside transfers. This would enable patterns such as autonomous agents, on-chain content moderation, and verifiable AI workflows without a trusted oracle bridge.

Open questions:

- **Determinism** — Reconciling non-deterministic model output with replayable state transitions across validators.
- **Verifiable AI** — Maturity of ZK-ML for proving inference correctness at acceptable cost.
- **Metering** — Gas-equivalent accounting for inference (FLOPs/energy, \$5.3) inside contract execution.
- **Reproducibility** — Model versioning and weight pinning so all validators evaluate identical models.
- **Consistency** — Cross-validator agreement when runtimes or model builds differ.

Status: research / future direction. **Phase 0 does not ship this.** Phase 1-2 exploration; depends on ZK-ML and determinism research before any productization. No committed date.

0x11.10 17.16 Anchoring to external networks

TET's own consensus provides liveness and finality; an open question is whether **periodic state anchoring** to established public networks (Bitcoin, Ethereum, IPFS/Filecoin) would add independent **long-term verifiability** and decentralization redundancy. Publishing a recurring commitment — e.g., a Merkle root of TET state — to a high-security external chain is a common pattern in modern L1 design and is

Plane	Participants	Implementation	Key provenance
Tmail / Files	browser ↔ browser (node is a blind relay)	'crystals-kyber-js' 1.1.2	**derived from the wallet BIP39 mnemonic** ('tmail_keys.ts')
AI inference E2EE	node ↔ worker	'pqcrypto-kyber' 0.8.1	ephemeral, '--keygen' CLI

Because the planes are disjoint, each can migrate independently — a coordinated cutover is **not** required.

Blocker: the Tmail/Files KEM keypair is mnemonic-derived, so changing the algorithm changes

every existing user's messaging identity and makes stored ciphertext undecryptable. Migration needs

a key-rotation/re-registration flow, or a chain reset that discards the old key directory. The

inference plane has no such constraint (keys are ephemeral).

Open questions:

particularly valuable for **audit-trail integrity**: anyone can later prove a TET state existed at a given time against the external anchor. This is distinct from, and complementary to, the cross-chain custody bridge open problem (\$17.6).

Open questions:

- **Frequency** — Anchor cadence, trading external fees against staleness of the latest verifiable checkpoint.
- **Commitment format** — Merkle root only vs a fuller state commitment.
- **Verifier tooling** — Lightweight tooling for third parties to check anchors independently.
- **Cost model** — Ethereum calldata/blob, Bitcoin OP_RETURN/inscription, and storage-network fees per anchor.

Status: research / future direction. **Phase 0 does not ship this.** Phase 1+ exploration; per-network effort is small-months scale but unscheduled. No commitment.

0x11.11 17.17 FIPS-203 ML-KEM migration (both E2EE planes)

Phase 0 ships **CRYSTALS-Kyber-768 (Round-3)**, not FIPS-203 ML-KEM-768. The two are

byte-incompatible: a Round-3 public key, ciphertext, and shared secret cannot be consumed by an

ML-KEM implementation, and vice versa. Earlier drafts of this document said "ML-KEM"; that was

inaccurate and is corrected throughout (2026-09-17).

Two independent planes use a KEM, and they never interoperate with each other:

- **Key migration** — re-registration flow vs discarding the directory at the Phase 1 genesis.

- **Dual-read window** — whether nodes should accept both algorithms during a transition, and for how long.

- **Library maturity** — FIPS-203 crate selection on both sides, and whether either has an audit.

- **Wire format** — the `mlkem_*` field names are legacy and currently misleading; renaming them is a protocol change and is deliberately deferred.

Status: open. Target **Phase 1 genesis**, where key migration is free because a new genesis is

```
| [ 0x12 ] 18. Comparison Table
```

```
--==[ TET NETWORK / WHITEPAPER v1.1 ]==-----
```

18. Comparison Table

Type	Source	Start date	End date	Time span	Reference	Page number
"1848"	US 1848, 1849	1848 (first issue)	1849 (last issue)	1 year	"1848 and 1849" (1848)	1848-1849
"1849"	US 1849, 1850	1849 (first issue)	1850 (last issue)	1 year	"1849 and 1850" (1849)	1849-1850
"1850"	US 1850, 1851	1850 (first issue)	1851 (last issue)	1 year	"1850 and 1851" (1850)	1850-1851
"1851"	US 1851, 1852	1851 (first issue)	1852 (last issue)	1 year	"1851 and 1852" (1851)	1851-1852
"1852"	US 1852, 1853	1852 (first issue)	1853 (last issue)	1 year	"1852 and 1853" (1852)	1852-1853
"1853"	US 1853, 1854	1853 (first issue)	1854 (last issue)	1 year	"1853 and 1854" (1853)	1853-1854
"1854"	US 1854, 1855	1854 (first issue)	1855 (last issue)	1 year	"1854 and 1855" (1854)	1854-1855
"1855"	US 1855, 1856	1855 (first issue)	1856 (last issue)	1 year	"1855 and 1856" (1855)	1855-1856
"1856"	US 1856, 1857	1856 (first issue)	1857 (last issue)	1 year	"1856 and 1857" (1856)	1856-1857
"1857"	US 1857, 1858	1857 (first issue)	1858 (last issue)	1 year	"1857 and 1858" (1857)	1857-1858
"1858"	US 1858, 1859	1858 (first issue)	1859 (last issue)	1 year	"1858 and 1859" (1858)	1858-1859
"1859"	US 1859, 1860	1859 (first issue)	1860 (last issue)	1 year	"1859 and 1860" (1859)	1859-1860
"1860"	US 1860, 1861	1860 (first issue)	1861 (last issue)	1 year	"1860 and 1861" (1860)	1860-1861
"1861"	US 1861, 1862	1861 (first issue)	1862 (last issue)	1 year	"1861 and 1862" (1861)	1861-1862
"1862"	US 1862, 1863	1862 (first issue)	1863 (last issue)	1 year	"1862 and 1863" (1862)	1862-1863
"1863"	US 1863, 1864	1863 (first issue)	1864 (last issue)	1 year	"1863 and 1864" (1863)	1863-1864
"1864"	US 1864, 1865	1864 (first issue)	1865 (last issue)	1 year	"1864 and 1865" (1864)	1864-1865
"1865"	US 1865, 1866	1865 (first issue)	1866 (last issue)	1 year	"1865 and 1866" (1865)	1865-1866
"1866"	US 1866, 1867	1866 (first issue)	1867 (last issue)	1 year	"1866 and 1867" (1866)	1866-1867
"1867"	US 1867, 1868	1867 (first issue)	1868 (last issue)	1 year	"1867 and 1868" (1867)	1867-1868
"1868"	US 1868, 1869	1868 (first issue)	1869 (last issue)	1 year	"1868 and 1869" (1868)	1868-1869
"1869"	US 1869, 1870	1869 (first issue)	1870 (last issue)	1 year	"1869 and 1870" (1869)	1869-1870
"1870"	US 1870, 1871	1870 (first issue)	1871 (last issue)	1 year	"1870 and 1871" (1870)	1870-1871
"1871"	US 1871, 1872	1871 (first issue)	1872 (last issue)	1 year	"1871 and 1872" (1871)	1871-1872
"1872"	US 1872, 1873	1872 (first issue)	1873 (last issue)	1 year	"1872 and 1873" (1872)	1872-1873
"1873"	US 1873, 1874	1873 (first issue)	1874 (last issue)	1 year	"1873 and 1874" (1873)	1873-1874
"1874"	US 1874, 1875	1874 (first issue)	1875 (last issue)	1 year	"1874 and 1875" (1874)	1874-1875
"1875"	US 1875, 1876	1875 (first issue)	1876 (last issue)	1 year	"1875 and 1876" (1875)	1875-1876
"1876"	US 1876, 1877	1876 (first issue)	1877 (last issue)	1 year	"1876 and 1877" (1876)	1876-1877
"1877"	US 1877, 1878	1877 (first issue)	1878 (last issue)	1 year	"1877 and 1878" (1877)	1877-1878
"1878"	US 1878, 1879	1878 (first issue)	1879 (last issue)	1 year	"1878 and 1879" (1878)	1878-1879
"1879"	US 1879, 1880	1879 (first issue)	1880 (last issue)	1 year	"1879 and 1880" (1879)	1879-1880
"1880"	US 1880, 1881	1880 (first issue)	1881 (last issue)	1 year	"1880 and 1881" (1880)	1880-1881
"1881"	US 1881, 1882	1881 (first issue)	1882 (last issue)	1 year	"1881 and 1882" (1881)	1881-1882
"1882"	US 1882, 1883	1882 (first issue)	1883 (last issue)	1 year	"1882 and 1883" (1882)	1882-1883
"1883"	US 1883, 1884	1883 (first issue)	1884 (last issue)	1 year	"1883 and 1884" (1883)	1883-1884
"1884"	US 1884, 1885	1884 (first issue)	1885 (last issue)	1 year	"1884 and 1885" (1884)	1884-1885
"1885"	US 1885, 1886	1885 (first issue)	1886 (last issue)	1 year	"1885 and 1886" (1885)	1885-1886
"1886"	US 1886, 1887	1886 (first issue)	1887 (last issue)	1 year	"1886 and 1887" (1886)	1886-1887
"1887"	US 1887, 1888	1887 (first issue)	1888 (last issue)	1 year	"1887 and 1888" (1887)	1887-1888
"1888"	US 1888, 1889	1888 (first issue)	1889 (last issue)	1 year	"1888 and 1889" (1888)	1888-1889
"1889"	US 1889, 1890	1889 (first issue)	1890 (last issue)	1 year	"1889 and 1890" (1889)	1889-1890
"1890"	US 1890, 1891	1890 (first issue)	1891 (last issue)	1 year	"1890 and 1891" (1890)	1890-1891
"1891"	US 1891, 1892	1891 (first issue)	1892 (last issue)	1 year	"1891 and 1892" (1891)	1891-1892
"1892"	US 1892, 1893	1892 (first issue)	1893 (last issue)	1 year	"1892 and 1893" (1892)	1892-1893
"1893"	US 1893, 1894	1893 (first issue)	1894 (last issue)	1 year	"1893 and 1894" (1893)	1893-1894
"1894"	US 1894, 1895	1894 (first issue)	1895 (last issue)	1 year	"1894 and 1895" (1894)	1894-1895
"1895"	US 1895, 1896	1895 (first issue)	1896 (last issue)	1 year	"1895 and 1896" (1895)	1895-1896
"1896"	US 1896, 1897	1896 (first issue)	1897 (last issue)	1 year	"1896 and 1897" (1896)	1896-1897
"1897"	US 1897, 1898	1897 (first issue)	1898 (last issue)	1 year	"1897 and 1898" (1897)	1897-1898
"1898"	US 1898, 1899	1898 (first issue)	1899 (last issue)	1 year	"1898 and 1899" (1898)	1898-1899
"1899"	US 1899, 1900	1899 (first issue)	1900 (last issue)	1 year	"1899 and 1900" (1899)	1899-1900

0x12.01 18.1 Layer 1 positioning

Factual summary from public project materials (2026-05). Where uncertain: **unknown / not stated**.

```
0x12.02 18.2 Secure messaging vs TET Tmail (Phase 0
target)
```

Capability	Signal	Telegram	Session	**TET Tmail (Phase 0)**	
libp2p/ decentralized transport	No		Yes	**Yes**	
Post-quantum (ML-DGA / KEM path)	No	No	No	**Yes** - Kyber Round-3; FIPS-203 ML-KEM migration scheduled for Phase 1 genesis (\$17.17)	
On-chain fee / audit metadata	No	No	No	**Yes**	
Time-lock delivery	No	No	No	**Yes** (stake-scheduled)	
Network burn-after-read	No	Limited (timer)	No	**Yes** (best-effort)	
2K anonymous sender + anchor audit	No	No	Partial (Session ID)	**Yes** (1 TET escrow)	

Differentiates **I1 + messaging** on two axes – not only consensus:

Do not over-claim: Rows assume 2026-09-15 ship targets pass AT-3..AT-5 on public testnet. Until then, table is **design intent**.

Sources (indicative): TET – this document + tet-core
+ SOVEREIGN OS PHASE0 SPEC.md; Signal/Telegram/Session
– public product docs. ^[6]

L1 caveat: TET testnet does not yet demonstrate planetary PoR scale, SP1 proofs, or closed-form η .

```
| [ 0x13 ] 19. Roadmap
```

```
--==[ TET NETWORK / WHITEPAPER v1.1 ]==-----
```

19. Roadmap

Sprint	Focus
S4	**1 Foundation** - 1 public seed (Hetzner EU), faucet 100 TET/day/IP, Docker node+UI, CI/CD, ops docs
S5	Tmail protocol + NFT + gossip
S6	Tmail EXEE + Win95 shell
S7	Time-lock + burn + Pin
S8	Anonymous Mode + 2K guest
S9	Files P2P
S10	Mini-apps (Calculator, Clock, Notes)
S11	QA + ship

0x13.01 19.1 Phase 0 – Sovereign OS + public testnet
(current)

Sprint **sequence** (see SOVEREIGN OS PHASE0 SPEC.md
§B.1): ^[7]

Deliverables:

- **Ship-able public testnet** (not UI-only): seed, faucet, docker compose up
- **Sovereign OS** (\$13): Wallet, Tmail (5 features), Files, mini-apps
- Thermodynamic pricing **approximation** (\$5.3)
- ML-DSA + Ed25519 hybrid wallets
- ZK-Court + Anonymous Tmail RISC0 path (dev ELF; CI stub where needed)

Explicit non-goals: Part II primitives (\$14-\$16),
mainnet freeze, SP1, cross-chain bridges,
productized AI Worker earn (Phase 0.5).

Target: 2026-09-15 public Phase 0 ship (feature freeze 2026-08-31). Operator / builder preview - not financial promotion.

0x13.02 19.2 Phase 1 – CAAC + economics hardening

- Full CAAC role automation; improved fingerprinting (\$17.5)
 - SP1 backend option (\$17.2)
 - VDF time-lock (\$17.8); stronger burn guarantees (\$17.9)
 - Founder vesting + REST API stability (\$17.3, \$17.10)
 - Capped slash classes using $\lambda \cdot R_{\text{expected}}$ (\$12.2)
- Optional Protocol Reserve funding via governance (\$11.4)
 - 2nd public seed + traffic SPOF mitigation
-
- 0x13.03 19.3 Phase 2 – Post-quantum fluid grid**
- η from attested power telemetry (\$5.5)
 - Federated learning / World Brain (\$14)
 - Sentient assets + Agent-Gate (\$15-\$16)
 - Scale-out toward edge participation thesis

[0x14] 20. Conclusion

---[TET NETWORK / WHITEPAPER v1.1]-----

20. Conclusion

TET Network is not an incremental L1 tweak. It couples **hardware-adaptive consensus**, **post-quantum signatures**, and an **energy-linked issuance philosophy** distinct from labor markets and centralized inference APIs. Version 1.1 makes the structure

honest: Part I is what we build and test; Part II is where the protocol could go; Part III is what still requires math, code, or both.

The testnet exists to find bugs like dual **genesis_hash** implementations before they become permanent mainnet failures. Engineers who can disprove a mechanism should do so on testnet, not in production.

[0x15] 21. A Call to Builders

---[TET NETWORK / WHITEPAPER v1.1]-----

21. A Call to Builders

Expertise sought (unchanged in spirit from v1.0):

- Rust systems programming (consensus, ledger, networking)
- libp2p at scale

- zkVM engineering (RISC Zero today; SP1 tomorrow)
- Applied cryptography (ML-DSA, hybrid protocols)
- Distributed ML infrastructure

Technical critique: **tetsteve@proton.me** (Subject: Core Builder Application)

[0x16] 22. References

---[TET NETWORK / WHITEPAPER v1.1]-----

22. References

Core consensus and cryptoeconomics

Zero-knowledge and post-quantum cryptography

Decentralized AI and networking

Thermodynamics and hardware security

Implementation annex (non-normative)

- tet-core/src/genesis.rs – canonical genesis_hash
- tet-core/src/vision/thermo_genesis.rs – Phase 0 thermodynamic estimate
- tet-core/src/ledger.rs – fees, burns, genesis allocation
- docs/WHITEPAPER_v1.0_GAPS.md – v1.0 vs code audit trail

End of Whitepaper v1.1 Draft – 2026-05-21

--=[REFERENCES]==--

1. [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*
2. [2] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.*
3. [3] Sompolinsky, Y., & Zohar, A. (2015). *Secure High-Rate Transaction Processing in Bitcoin.*
4. [4] Buterin, V., & Griffith, V. (2017). *Casper the Friendly Finality Gadget.*
5. [5] Ben-Sasson, E., et al. (2014). *SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge.*
6. [6] RISC Zero Team. (2023). *RISC Zero zkVM.*
7. [7] Succinct Labs. (2024). *SP1 zkVM.*
8. [8] NIST. (2024). *FIPS 204: ML-DSA.*
9. [9] Rao, Y. (2021). *Bittensor: A Peer-to-Peer Intelligence Market.*
10. [10] Borzunov, A., et al. (2022). *Petals: Collaborative Inference and Fine-tuning of Large Models.*
11. [11] AI@Meta. (2024). *Llama 3 Model Card.*
12. [12] Protocol Labs. (2019). *libp2p.*
13. [13] McMahan, B., et al. (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data.*
14. [14] Landauer, R. (1961). *Irreversibility and Heat Generation in the Computing Process.*
15. [15] Bennett, C. H. (1982). *The Thermodynamics of Computation.*
16. [16] Suh, G. E., & Devadas, S. (2007). *Physical Unclonable Functions for Device Authentication.*